In [1]:
```
import seaborn as sns
import matplotlib as plt
import pandas as pd
import numpy as np
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

In [2]:
```
data = pd.read_csv(r"D:\car-mpg.csv")
```

In [3]:
```
data
```

Out[3]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino
...	...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86	2790	15.6	82	1	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	1	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	1	chevy s-10

398 rows × 10 columns

In [4]:
```
data.head()
```

Out[4]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

In [5]:
```
data.shape
```

Out[5]: (398, 10)

In [6]:
```
data.isnull().sum()
```

Out[6]:
```
mpg         0
cyl         0
disp        0
hp          0
wt          0
acc         0
yr          0
origin      0
car_type    0
car_name    0
dtype: int64
```

In [7]:
```
data.head(1)
```

Out[7]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu

In [8]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 10 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   mpg       398 non-null    float64
 1   cyl       398 non-null    int64  
 2   disp      398 non-null    float64
 3   hp        398 non-null    object 
 4   wt        398 non-null    int64  
 5   acc       398 non-null    float64
 6   yr        398 non-null    int64  
 7   origin    398 non-null    int64  
 8   car_type  398 non-null    int64  
 9   car_name  398 non-null    object 
dtypes: float64(3), int64(5), object(2)
memory usage: 31.2+ KB
```

In [9]:
```
data = data.drop(['car_name'], axis=1)
data['origin'] = data['origin'].replace({1:'america', 2: 'europe', 3:'asis'})
data = pd.get_dummies(data, columns=['origin'], dtype = int)

import numpy as np
data  = data.replace('?', np.nan)
data = data.apply(pd.to_numeric, errors='ignore')

data = data.apply(lambda x:x.fillna(x.median()) if x.dtype != 'object' else x, axis
```

```
C:\Users\3star\AppData\Local\Temp\ipykernel_37976\1534430809.py:7: FutureWarning: er
rors='ignore' is deprecated and will raise in a future version. Use to_numeric witho
ut passing `errors` and catch exceptions explicitly instead
  data = data.apply(pd.to_numeric, errors='ignore')
```

In [10]:
```
data.isnull().sum()
```

Out[10]:
```
mpg              0
cyl              0
disp             0
hp               0
wt               0
acc              0
yr               0
car_type         0
origin_america   0
origin_asis      0
origin_europe    0
dtype: int64
```

In [11]:
```
data.head(1)
```

Out[11]:

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asis	origin_eur
0	18.0	8	307.0	130.0	3504	12.0	70	0	1	0	

In [12]:
```
x = data.drop(['mpg'], axis = 1)
y = data[['mpg']]
```

In [13]:
```
x.head(3)
```

Out[13]:

	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asis	origin_europe
0	8	307.0	130.0	3504	12.0	70	0	1	0	0
1	8	350.0	165.0	3693	11.5	70	0	1	0	0
2	8	318.0	150.0	3436	11.0	70	0	1	0	0

In [14]:
```
x_s = preprocessing.scale(x)
x_s
```

Out[14]:
```
array([[ 1.49819126,  1.0906037 ,  0.67311762, ...,  0.77355903,
        -0.49764335, -0.46196822],
       [ 1.49819126,  1.5035143 ,  1.58995818, ...,  0.77355903,
        -0.49764335, -0.46196822],
       [ 1.49819126,  1.19623199,  1.19702651, ...,  0.77355903,
        -0.49764335, -0.46196822],
       ...,
       [-0.85632057, -0.56103873, -0.53187283, ...,  0.77355903,
        -0.49764335, -0.46196822],
       [-0.85632057, -0.70507731, -0.66285006, ...,  0.77355903,
        -0.49764335, -0.46196822],
       [-0.85632057, -0.71467988, -0.58426372, ...,  0.77355903,
        -0.49764335, -0.46196822]], shape=(398, 10))
```

In [15]:
```
y_s = preprocessing.scale(y)
y_s
```

Out[15]:
```
array([[-7.06438701e-01],
       [-1.09075062e+00],
       [-7.06438701e-01],
       [-9.62646649e-01],
       [-8.34542675e-01],
       [-1.09075062e+00],
       [-1.21885460e+00],
       [-1.21885460e+00],
       [-1.21885460e+00],
       [-1.09075062e+00],
       [-1.09075062e+00],
       [-1.21885460e+00],
       [-1.09075062e+00],
       [-1.21885460e+00],
       [ 6.21851453e-02],
       [-1.94022803e-01],
       [-7.06438701e-01],
       [-3.22126778e-01],
       [ 4.46497068e-01],
       [ 3.18393094e-01],
       [ 1.90289120e-01],
       [ 6.21851453e-02],
       [ 1.90289120e-01],
       [ 3.18393094e-01],
       [-3.22126778e-01],
       [-1.73127050e+00],
       [-1.73127050e+00],
       [-1.60316652e+00],
       [-1.85937447e+00],
       [ 4.46497068e-01],
       [ 5.74601043e-01],
       [ 1.90289120e-01],
       [ 1.90289120e-01],
       [-5.78334726e-01],
       [-9.62646649e-01],
       [-8.34542675e-01],
       [-5.78334726e-01],
       [-7.06438701e-01],
       [-1.21885460e+00],
       [-1.21885460e+00],
       [-1.21885460e+00],
       [-1.21885460e+00],
       [-1.47506255e+00],
       [-1.34695857e+00],
       [-1.34695857e+00],
       [-7.06438701e-01],
       [-1.94022803e-01],
       [-5.78334726e-01],
       [-7.06438701e-01],
       [-6.59188290e-02],
       [ 5.74601043e-01],
       [ 8.30808991e-01],
       [ 8.30808991e-01],
       [ 9.58912966e-01],
       [ 1.47132886e+00],
       [ 4.46497068e-01],
```

```
[ 3.18393094e-01],
[ 6.21851453e-02],
[ 1.90289120e-01],
[-6.59188290e-02],
[-4.50230752e-01],
[-3.22126778e-01],
[-1.34695857e+00],
[-1.21885460e+00],
[-1.09075062e+00],
[-1.21885460e+00],
[-8.34542675e-01],
[-1.60316652e+00],
[-1.34695857e+00],
[-1.47506255e+00],
[-1.34695857e+00],
[-5.78334726e-01],
[-1.09075062e+00],
[-1.34695857e+00],
[-1.34695857e+00],
[-1.21885460e+00],
[-7.06438701e-01],
[-1.94022803e-01],
[-3.22126778e-01],
[ 3.18393094e-01],
[-1.94022803e-01],
[ 5.74601043e-01],
[-6.59188290e-02],
[ 5.74601043e-01],
[ 4.46497068e-01],
[-1.34695857e+00],
[-1.21885460e+00],
[-1.34695857e+00],
[-1.21885460e+00],
[-1.09075062e+00],
[-1.47506255e+00],
[-1.34695857e+00],
[-1.34695857e+00],
[-1.21885460e+00],
[-1.34695857e+00],
[-1.47506255e+00],
[-1.34695857e+00],
[-7.06438701e-01],
[-9.62646649e-01],
[-7.06438701e-01],
[-7.06438701e-01],
[-6.59188290e-02],
[ 3.18393094e-01],
[-1.60316652e+00],
[-1.47506255e+00],
[-1.34695857e+00],
[-1.47506255e+00],
[-7.06438701e-01],
[-4.50230752e-01],
[-3.22126778e-01],
[-1.94022803e-01],
[-7.06438701e-01],
```

```
[-5.78334726e-01],
[-3.22126778e-01],
[ 3.18393094e-01],
[-1.09075062e+00],
[-9.62646649e-01],
[ 7.02705017e-01],
[ 6.21851453e-02],
[-4.50230752e-01],
[-5.78334726e-01],
[-1.09075062e+00],
[ 6.21851453e-02],
[-4.50230752e-01],
[-1.60316652e+00],
[-4.50230752e-01],
[-3.22126778e-01],
[-5.78334726e-01],
[-1.09075062e+00],
[ 9.58912966e-01],
[ 3.18393094e-01],
[ 1.08701694e+00],
[ 1.90289120e-01],
[-9.62646649e-01],
[-9.62646649e-01],
[-7.06438701e-01],
[-9.62646649e-01],
[-1.34695857e+00],
[-1.21885460e+00],
[-1.21885460e+00],
[-1.21885460e+00],
[ 7.02705017e-01],
[ 3.18393094e-01],
[ 3.18393094e-01],
[ 9.58912966e-01],
[ 1.08701694e+00],
[ 5.74601043e-01],
[ 6.21851453e-02],
[ 3.18393094e-01],
[ 6.21851453e-02],
[ 3.18393094e-01],
[ 9.58912966e-01],
[-5.78334726e-01],
[-7.06438701e-01],
[-1.09075062e+00],
[-1.09075062e+00],
[-9.62646649e-01],
[-1.09075062e+00],
[-9.62646649e-01],
[-1.21885460e+00],
[-8.34542675e-01],
[-9.62646649e-01],
[-1.09075062e+00],
[-7.06438701e-01],
[-3.22126778e-01],
[-4.50230752e-01],
[-1.34695857e+00],
[ 7.02705017e-01],
```

```
[-6.59188290e-02],
[-4.50230752e-01],
[-6.59188290e-02],
[ 6.21851453e-02],
[ 1.90289120e-01],
[ 6.21851453e-02],
[-7.06438701e-01],
[ 7.02705017e-01],
[-5.78334726e-01],
[-6.59188290e-02],
[-6.59188290e-02],
[-1.94022803e-01],
[ 1.90289120e-01],
[ 1.21512091e+00],
[ 5.74601043e-01],
[ 1.90289120e-01],
[ 1.90289120e-01],
[ 3.18393094e-01],
[ 4.46497068e-01],
[-7.70490688e-01],
[-9.62646649e-01],
[-1.02669864e+00],
[-1.15480261e+00],
[-1.94022803e-01],
[-1.94022803e-01],
[ 6.21851453e-02],
[-1.29970816e-01],
[ 7.02705017e-01],
[ 1.26237132e-01],
[ 7.02705017e-01],
[ 1.21512091e+00],
[-4.50230752e-01],
[-7.06438701e-01],
[-6.42386713e-01],
[-7.70490688e-01],
[ 7.66757004e-01],
[ 1.08701694e+00],
[ 5.74601043e-01],
[ 3.82445081e-01],
[-4.50230752e-01],
[-1.34695857e+00],
[-5.78334726e-01],
[-5.78334726e-01],
[-8.98594662e-01],
[-8.98594662e-01],
[-1.34695857e+00],
[-1.34695857e+00],
[-1.34695857e+00],
[ 1.02296495e+00],
[ 8.30808991e-01],
[ 1.59943284e+00],
[ 2.54341107e-01],
[ 1.27917290e+00],
[-7.70490688e-01],
[-8.34542675e-01],
[-1.02669864e+00],
```

```
[-1.09075062e+00],
[-7.70490688e-01],
[-3.86178765e-01],
[-5.78334726e-01],
[-6.42386713e-01],
[-9.62646649e-01],
[-1.02669864e+00],
[-1.02669864e+00],
[-9.62646649e-01],
[ 7.02705017e-01],
[ 1.26237132e-01],
[ 3.18393094e-01],
[ 2.54341107e-01],
[ 8.94860978e-01],
[ 1.27917290e+00],
[ 8.30808991e-01],
[ 8.94860978e-01],
[-1.94022803e-01],
[-2.58074790e-01],
[-2.58074790e-01],
[ 2.50897106e+00],
[ 1.61224323e+00],
[ 1.18950012e+00],
[ 2.03498635e+00],
[ 1.61224323e+00],
[-4.63041149e-01],
[-5.27093137e-01],
[-4.24609957e-01],
[-5.52713931e-01],
[-3.86178765e-01],
[-4.24609957e-01],
[ 2.03099517e-01],
[-3.86178765e-01],
[-5.27093137e-01],
[-3.73368367e-01],
[-3.47747573e-01],
[-6.29576316e-01],
[-6.93628303e-01],
[-5.52713931e-01],
[-7.44869893e-01],
[-6.93628303e-01],
[-7.70490688e-01],
[ 8.30808991e-01],
[ 5.10549055e-01],
[ 4.72117863e-01],
[ 9.46102568e-01],
[-3.09316380e-01],
[-4.02980341e-02],
[ 3.65643505e-02],
[ 4.93747479e-02],
[-4.11799560e-01],
[-8.34542675e-01],
[-2.45264393e-01],
[-9.37025854e-01],
[ 1.02296495e+00],
[ 7.66757004e-01],
```

```
[-2.58074790e-01],
[-4.75851547e-01],
[-1.55591611e-01],
[-4.24609957e-01],
[-3.73368367e-01],
[-8.34542675e-01],
[-7.57680290e-01],
[-8.98594662e-01],
[-6.80817906e-01],
[-8.47353072e-01],
[-1.02669864e+00],
[-5.52713931e-01],
[-6.42386713e-01],
[ 1.07420654e+00],
[ 1.35603529e+00],
[ 1.56100164e+00],
[ 4.97738658e-01],
[ 2.41530709e-01],
[-6.59188290e-02],
[ 4.72117863e-01],
[ 4.93747479e-02],
[ 1.36884568e+00],
[ 1.40727688e+00],
[ 1.06139615e+00],
[ 1.76596800e+00],
[ 6.25842632e-01],
[ 6.77084222e-01],
[ 4.20876273e-01],
[ 1.27917290e+00],
[ 2.30400470e+00],
[ 1.86845118e+00],
[ 1.09982734e+00],
[ 1.75315761e+00],
[ 5.74601043e-01],
[ 3.69634684e-01],
[ 1.00616338e-01],
[-5.65524329e-01],
[ 1.38165608e+00],
[ 8.05188196e-01],
[ 9.97344158e-01],
[ 1.72753681e+00],
[ 1.11263773e+00],
[ 2.95733497e+00],
[ 5.61790645e-01],
[ 2.21433191e+00],
[ 2.66269582e+00],
[ 2.54740225e+00],
[ 1.65067443e+00],
[ 8.30808991e-01],
[ 2.70112702e+00],
[ 2.22714231e+00],
[ 1.31760409e+00],
[ 8.05188196e-01],
[ 1.17668972e+00],
[ 2.37539530e-02],
[ 1.47132886e+00],
```

```
[ 1.09435556e-02],
[ 1.13825853e+00],
[ 4.72117863e-01],
[ 3.95255479e-01],
[ 2.92772299e-01],
[-1.86684184e-03],
[ 8.30808991e-01],
[ 1.99655516e+00],
[ 1.98374476e+00],
[ 1.48413926e+00],
[ 1.12544813e+00],
[ 1.72753681e+00],
[ 1.81720959e+00],
[ 1.35603529e+00],
[ 1.43289767e+00],
[ 1.39446648e+00],
[ 8.17998594e-01],
[ 1.21512091e+00],
[ 1.40727688e+00],
[ 1.30479370e+00],
[ 1.13825853e+00],
[ 1.20231052e+00],
[ 1.03577535e+00],
[ 5.87411440e-01],
[ 9.20481773e-01],
[ 2.41530709e-01],
[ 8.78059402e-02],
[-1.42781214e-01],
[ 3.95255479e-01],
[-4.24609957e-01],
[-7.57680290e-01],
[ 5.74601043e-01],
[ 4.46497068e-01],
[ 1.34322489e+00],
[ 9.58912966e-01],
[ 7.02705017e-01],
[ 4.46497068e-01],
[ 6.21851453e-02],
[-6.59188290e-02],
[ 1.59943284e+00],
[ 1.72753681e+00],
[ 9.58912966e-01],
[ 1.85564079e+00],
[ 1.59943284e+00],
[ 1.59943284e+00],
[ 1.59943284e+00],
[ 1.34322489e+00],
[ 1.85564079e+00],
[ 1.08701694e+00],
[ 1.85564079e+00],
[ 1.90289120e-01],
[ 1.85564079e+00],
[ 3.18393094e-01],
[-1.94022803e-01],
[ 1.08701694e+00],
[ 1.59943284e+00],
```

```
       [ 4.46497068e-01],
       [ 4.46497068e-01],
       [ 2.62426463e+00],
       [ 1.08701694e+00],
       [ 5.74601043e-01],
       [ 9.58912966e-01]])
```

In [16]:
```
x_s = pd.DataFrame(x_s, columns=x.columns)
y_s = pd.DataFrame(y_s, columns=y.columns)
```

In [17]:
```
x_s
```

Out[17]:

	cyl	disp	hp	wt	acc	yr	car_type	origin_ame
0	1.498191	1.090604	0.673118	0.630870	-1.295498	-1.627426	-1.062235	0.773
1	1.498191	1.503514	1.589958	0.854333	-1.477038	-1.627426	-1.062235	0.773
2	1.498191	1.196232	1.197027	0.550470	-1.658577	-1.627426	-1.062235	0.773
3	1.498191	1.061796	1.197027	0.546923	-1.295498	-1.627426	-1.062235	0.773
4	1.498191	1.042591	0.935072	0.565841	-1.840117	-1.627426	-1.062235	0.773
...	...	...	...	...	...	...	...	
393	-0.856321	-0.513026	-0.479482	-0.213324	0.011586	1.621983	0.941412	0.773
394	-0.856321	-0.925936	-1.370127	-0.993671	3.279296	1.621983	0.941412	-1.292
395	-0.856321	-0.561039	-0.531873	-0.798585	-1.440730	1.621983	0.941412	0.773
396	-0.856321	-0.705077	-0.662850	-0.408411	1.100822	1.621983	0.941412	0.773
397	-0.856321	-0.714680	-0.584264	-0.296088	1.391285	1.621983	0.941412	0.773

398 rows × 10 columns

In [18]:
```
y_s
```

Out[18]:

	mpg
0	-0.706439
1	-1.090751
2	-0.706439
3	-0.962647
4	-0.834543
...	...
393	0.446497
394	2.624265
395	1.087017
396	0.574601
397	0.958913

398 rows × 1 columns

In [19]:
```
x_train, x_test, y_train, y_test = train_test_split(x_s, y_s, test_size=0.3, random
```

In [20]:
```
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(x_test.shape)
```

```
(278, 10)
(120, 10)
(278, 1)
(120, 10)
```

In [21]:
```
len(data)
```

Out[21]:
```
398
```

In [22]:
```
data.columns
```

Out[22]:
```
Index(['mpg', 'cyl', 'disp', 'hp', 'wt', 'acc', 'yr', 'car_type',
       'origin_america', 'origin_asis', 'origin_europe'],
      dtype='object')
```

In [23]:
```
regression_model = LinearRegression()
regression_model.fit(x_train, y_train)

for idx, col_name in enumerate(x_train.columns):
    print("the coef for {} is {}".format(col_name, regression_model.coef_[0][idx]))

print("=====")

intercept = regression_model.intercept_[0]
```

```
print("The intercept is {}".format(intercept))

print("=====")
```

```
the coef for cyl is 0.2474447975894667
the coef for disp is 0.2883821544609869
the coef for hp is -0.18990342687152895
the coef for wt is -0.6732229065111772
the coef for acc is 0.0675450154068817
the coef for yr is 0.3446364072117273
the coef for car_type is 0.31491491540037625
the coef for origin_america is -0.07682943694882868
the coef for origin_asis is 0.0633604889661999
the coef for origin_europe is 0.031283357351475166
=====
The intercept is -0.01950046762401743
=====
```

RIDGE MODEL

In [24]:
```
ridge_model = Ridge(alpha=0.4)
ridge_model.fit(x_train, y_train)

print("Ridge model coef: {}".format(ridge_model.coef_))
```

```
Ridge model coef: [ 0.2431613   0.27536624 -0.18977371 -0.66177905  0.06534252  0.34
374093
  0.31063723 -0.07629922  0.06332619  0.03064529]
```

LASSO MODEL

In [27]:
```
lasso_model = Lasso(alpha=0.1)
lasso_model.fit(x_train, y_train)

print("Lasson model coef: {}".format(lasso_model.coef_))
```

```
Lasson model coef: [-0.         -0.         -0.06203044 -0.48363379  0.          0.2
7163751
  0.09620861 -0.03490256  0.          0.        ]
```

In [28]:
```
print(regression_model.score(x_train, y_train))
print(regression_model.score(x_test, y_test))
print("***")
print(ridge_model.score(x_train, y_train))
print(ridge_model.score(x_test, y_test))
print("***")
print(lasso_model.score(x_train, y_train))
print(lasso_model.score(x_test, y_test))
print("***")
```

```
0.836163800114943
0.8439452810748137
***
0.8361431007135054
0.84372894771266
***
0.7994535676270829
0.81026554865651
***
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: